User Modeling and User-Adapted Interaction (2024) 34:691–728
https://doi.org/10.1007/s11257-023-09379-6

Hybrid session-aware recommendation with feature-based models

Josef Bauer[1] · **Dietmar Jannach**[1]

Received: 28 December 2022 / Accepted in revised form: 27 July 2023 / Published online: 6 October 2023

Abstract

Session-based recommender systems model the interests of users based on their browsing behavior with the goal of making suitable item suggestions in an ongoing usage session. Most existing work in this growing research area make only use of the most recent observed interactions for each user, and they typically solely rely on user–item interaction data (e.g., click events) for interest modeling. Thus, they do not leverage important forms of other information which are commonly available in practical settings. In this work, we therefore propose a hybrid approach for *personalized* session-based (*"session-aware"*) recommendation, which *(i)* is able to take into account various types of side information as model features and which *(ii)* can be combined with existing session-based (or session-aware) recommendation models. Technically, our approach is based on stacking several session-based modeling approaches with efficient machine learning methods for tabular data, in our case using Gradient Boosting Machines (GBMs). We successfully evaluated our approach (named *HySAR*) on two public e-commerce datasets. Specifically, we also demonstrate the effectiveness of a number of novel model features that we engineered in the course of this research. These features, which were mostly unexplored in previous works, relate to various types of information related to the users, their actions, the items, as well as contextual session characteristics. Different existing recommendation approaches and further problem specific features can be easily added in our generic method to improve recommendations.

Keywords Recommender systems · Session-based recommendation · Hybrid approaches · Tabular data

✉ Josef Bauer
josef.b.bauer@gmail.com

Dietmar Jannach
dietmar.jannach@aau.at

[1] University of Klagenfurt, Klagenfurt, Austria

1 Introduction

Recommender Systems are decision support systems with the purpose of helping users finding items that are relevant to them within a rich set of options. These systems are nowadays part of many modern online services, e.g., on e-commerce or media streaming sites, where they often create substantial business value for providers (Jannach and Jugovac 2019). Traditionally, recommender systems were designed to model only long-term preference of users. In session-based recommendation scenarios, in contrast, the goal is to make item suggestions during an ongoing shopping or browsing session. In such situations, short-term intents and user needs have to be estimated from the interactions of the current session and are often much more important than long-term preferences (Jannach et al. 2017, 2021; Quadrana et al. 2018; Wang et al. 2021).

Being able to make accurate session-based recommendations using only a few recent observed interactions is a highly relevant problem in practice. In e-commerce settings, for example, many shop visitors might be entirely anonymous, either because they are new customers or because they or not logged in. In such cases, no long-term preference information is available at all. But even in cases the customer is already known and logged in, it often is highly important to make recommendations that match the user's current short-term shopping *intents*, which may change from session to session (Jannach et al. 2017). Due to the practical relevance of the problem, various technical approaches were proposed in the past few years not only in the area of e-commerce, but also for several other application domains such as music, news, or movies (Garcin et al. 2013; Hidasi et al. 2016a; Kouki et al. 2020; Liu et al. 2018; Ren et al. 2019; Shani et al. 2005).

Session-based recommendation techniques date back more than twenty years, see, e.g., Mobasher et al. (2002) for an early work. Research interest in the area however only grew substantially after the ACM RecSys Challenge in 2015 (Ben-Shimon et al. 2015). In the context of this challenge a large dataset of anonymous shopping sessions was released, which contained click events collected from an online retailer. While the availability of this and similar datasets strongly fueled research in this area, the information in such datasets is in many cases limited to certain types of recorded interactions between online users and items, in particular *view* and *purchase* events. As a result, the majority of published research in session-based recommendations is focused on developing models that solely rely on this type of *collaborative* information.

In practice, in contrast, additional information about the items, e.g., meta-data or the actual *content* in case of media items, is almost always available. In traditional recommendation scenarios, a large number of *hybrid* techniques were proposed during the last three decades (Burke 2002). Such techniques usually combine collaborative and content-based approaches for improved recommendations. Very commonly, such hybrids were designed for *user cold-start* situations, i.e., situations where little is known about the users. User cold start is in fact *the* central characteristic of session-based recommendation problems. Nonetheless, research on content-based or hybrid session-based recommendation techniques is still comparably limited today. This seems surprising, in particular as studies like (Jannach et al. 2017) indicate that

even simple heuristics like recommending items from the same category as the last viewed one can be helpful in an e-commerce shop.

Examples of works that try to combine different types of information in hybrid session-based recommenders include (Gabriel et al. 2019) or (Hidasi et al. 2016b), which rely on images, textual data, or category information for improved recommendations. Often however, only one or a few types of side information are considered in such approaches. Moreover, the integration of side information into complex session-based recommendation models can be challenging, leading to a technical integration approach that is tailored to the specific model that operates on the basis of collaborative information, e.g., a particular recurrent neural network (RNN) architecture.

In terms of combining long-term preference models with the most recently observed interactions, only a few works were proposed in the last years, e.g., Liang et al. (2018); Phuong et al. (2019); Quadrana et al. (2017); Ruocco et al. (2017); Ying et al. (2018). However, a recent study (Latifi et al. 2021) showed that these *personalized* session-based (or: "session-aware") methods are actually not effective in incorporating long-term preference signals, and can be outperformed by purely session-based techniques, which do not make use of the available long-term information about users.

In this work, we therefore propose an approach to leverage long-term preference information and different types of side information in a generic approach to session-aware recommendation settings. Specifically, our approach has two phases. In the first phase, a set of items is pre-selected and scored using one or more baseline item rankings. Such baseline rankings can be done solely based on collaborative information and existing (content-agnostic) session-based recommendation techniques like GRU4Rec (Hidasi et al. 2016a). In the second phase, we integrate the baseline scores with engineered features, which—among other aspects—capture long-term preference information. We do this by formulating the problem in such a way that efficient machine learning methods for tabular data—in our case Gradient Boosting Machines (GBMs)—can be used.

We demonstrate the effectiveness of our approach for the e-commerce domain. Experiments with two public datasets that contain item meta-data show that the incorporation of side information and long-term preference information with our approach helps to significantly increase prediction accuracy. In this context, we also evaluate a variety of features that have not been previously explored in the context of session-aware recommendation in e-commerce. Most of the features that were engineered in our generic approach are general and could be used for other e-commerce datasets or related applications. We analyze the importance of these features based on Shapley values, which also allows us to address interpretability aspects. To ensure reproducibility, we share the source code and links to the datasets used in our experiments online.[1] While our experiments so far are limited to the e-commerce domain, we emphasize that our overall method is generic and not tied to a particular domain.

We organize the paper as follows. After reviewing previous work in Sect. 2, we introduce the problem setting and general technical solution approach in Sect. 3. Details of our method are described in Sect. 4. The experimental results are reported in Sect. 5,

[1] https://github.com/jo-bauer/hysar.

followed by a feature analysis in Sect. 6 and an ablation study in Sect. 7. The paper ends with a discussion of our findings and an outlook on future works.

2 Background and related work

According to Quadrana et al. (2018), session-based recommendation can be seen as a subclass of the more general class of *sequence-aware* recommendation problems. Specifically, the authors of Quadrana et al. (2018) differentiate between the following problem classes:

- Session-*based* recommendation, where users are anonymous and we only know the interactions of a user in the ongoing session.
- Session-*aware* recommendation, where the data is also organized in sessions, but users are not anonymous and we additionally have knowledge about previous sessions of the current user. This is the focus of our present work. However, we want to point out that our approach can also be used for pure session-*based* recommendation, simply by not incorporating any features that depend on specific user information, as will become clear in Sect. 4.3.
- Sequential recommendation, where users are also not anonymous and interactions are also time-ordered, but the interactions are not organized in sessions. Recent examples of such approaches include Bert4REC (Sun et al. 2019) or SASRec (Kang and McAuley 2018).

Overall, while these approaches operate on different types of data, they share the common goal, namely the recommendation of items that are expected to match the current or latest intents, needs, or interests of the user. Amazon.com's "*Customers who bought …also bought"* feature could be seen as a very simple implementation of such an approach, which is non-personalized and only takes the last user interaction into account. A multitude of more elaborate methods were proposed over the years, which, from a machine learning perspective, aim to predict a user's next interaction with an item given a sequence of past interactions. In an early work, Mobasher et al. (2002), for example, used sequential patterns to make next-page browsing recommendations. Later, Shani et al. (2005) viewed recommendation as a sequential optimization problem and modeled user sessions as Markov decision processes. At around the same time, Ragno et al. (2005) explored an approach for session-based music recommendation based on item similarities. Music recommendation was also the focus of Hariri et al. (2012), who relied on collaborative information and a nearest-neighbor technique in combination with latent topic information. A more complex model for music recommendation based on metric embedding was proposed in Chen et al. (2012).

Up to about the year 2015, the literature on session-based recommendation was rather sparse, with some research published on non-public datasets from time to time, e.g., Garcin et al. (2013); Jannach et al. (2015); Tavakol and Brefeld (2014a). Since 2015, however, a large number of technical approaches for session-based recommendation were published, see (Wang et al. 2021) for a recent survey. This development was fueled both by the availability of public datasets and the general boom in deep learning for recommender systems that started at this time. GRU4Rec was probably

the first neural recommendation method designed specifically for session-based recommendation (Hidasi et al. 2016a). This widely known method is based on an adapted recurrent neural network architecture that relies on Gated Recurrent Units that were introduced a year before and which, among other aspects, help avoid the problem of vanishing gradients. Several improvements were later suggested for GRU4Rec, including alternative loss functions that have shown to lead to significantly better performance. Since then, various other types of neural network architectures were considered for session-based recommendation, including combinations of RNNs with attention layers, convolutional layers, memory networks, graph neural networks or variational autoencoders, e.g., Li et al. (2017); Liu et al. (2018); Sachdeva et al. (2019); Wang et al. (2019); Wu et al. (2019); Yu et al. (2020); Yuan et al. (2019).

In the method proposed in this work, any of these complex models can be used in the first phase, which has the goal to make a pre-selection of relevant items. In our experiments evaluation, see Sect. 5, we try out two alternative methods from different families of approaches. First, we use the latest version of GRU4Rec, which is widely used and which we found to be still very competitive in a recent performance evaluation (Ludewig et al. 2021). In addition, we run experiments in which we use a session-based nearest-neighbor method (S-SKNN) for item pre-selection. Again, according to previous work, such methods despite their conceptual simplicity often lead to highly competitive results, and in many cases they even outperform more complex neural models, cf. Ludewig et al. (2021).

Notable works on *session-aware* recommendation techniques started to appear around 2017. In Quadrana et al. (2017), the authors built upon the GRU4Rec system and trained two recurrent neural networks, where one GRU layer is used to model the current session and the other one models the user information across sessions. A similar RNN-based approach was proposed in Ruocco et al. (2017). The work described in Jannach et al. (2017), in contrast, is based on a two-phase reranking approach similar to our present work.[2] There, a session-based technique based on nearest neighbors is used to pre-select a set of candidate items. In the second phase, various prediction features were engineered, and a neural network architecture finally was found to be the most effective method to combine the features in the prediction process. Ying et al. (2018) later on proposed a neural model that relies on two-layer hierarchical attention network. In this model, the first layer learns long-term user preferences and the second layer combines this model with the embeddings of the items of the current session. Differently from that, the NCSF method proposed in Liang et al. (2018) combines three components, a historical session encoder, an encoder for the current session, and a joint-context encoder to combine the different components. In Phuong et al. (2019), finally, also RNNs are used to model short-term and long-term models, and different ways are proposed to combine the models, e.g., with a gating mechanisms that allows to make the contribution of each component fixed or adaptive. Overall,

[2] Reranking techniques are widely used in the recommender systems literature, e.g., to increase the diversity of item lists or to combine various models, e.g., in the context of similar-item recommendations (Adomavicius and Kwon 2011; Sánchez et al. 2020). Note that our method uses the selected items of several (in our experiments two) baseline recommender methods and incorporates their predicted ranks as features among many other types of features. It can therefore be considered as an ensembling approach rather than a reranking of one algorithm, although it can be used with just one baseline recommender as well.

however, while various neural models were proposed in recent years, a recent study in Latifi et al. (2021) as mentioned above indicated that these models are actually not very effective. For example, the latest version of GRU4Rec turned out to be more effective than combining two GRU4Rec models as proposed in Quadrana et al. (2017). Moreover, session-based nearest neighbor methods were consistently better than all session-aware methods discussed here. In our present work, we therefore do not compare our proposed method with these models but only with methods for session-based recommendation.

The session-based and session-aware methods discussed so far are purely collaborative and do not rely on any side information. An earlier, non-neural approach that does rely on side information for e-commerce recommendations was proposed in Tavakol and Brefeld (2014b). Technically, their approach relies on factored Markov decision processes. Regarding the used side information, the approach is primarily focused on topic (category) detection, which is then used for recommendation. Since the state space can become very big, its applicability to huge datasets is however limited. An early deep learning approach that considers side information is presented in Hidasi et al. (2016b). In their work, Hidasi et al. incorporate image and text data and tried several parallel RNN architectures. Training a separate RNN for every feature and combining them in the end however worked best according to their respective experiments. Differently from our work, the work in Hidasi et al. (2016b) is however somewhat limited in the form of the features that can be taken into account. The work presented in Song and Lee (2018) also uses RNNs and user-based features, but only static ones. Dynamic features that change over time can not be included because of the underlying one-hot encoding of categorical features that is used in the approach. Similar limitations apply to Phuong et al. (2018), which incorporates user-based context information within RNNs in the form of embeddings, and Zhu et al. (2020), which only uses category information to model user intents. Likewise, Song et al. (2019) only uses n-gram features of the items, but do not consider other features. In our approach, in contrast, a multitude of features can be incorporated, in particular dynamic, time-dependent ones, for example, how the users interact with items and categories in the current session and in the past.

The method presented in de Souza et al. (2019), finally, is based on a hybrid RNN method and, like our work, uses more features. The approach is however specifically tailored to a certain news recommendation problem. For instance, their architecture integrates specific components for natural language processing and the creation of embeddings for the news items. Moreover, central factors like recency and general popularity are particularly important in news recommendation scenarios. Our work, in contrast, mainly focuses on e-commerce applications as well as different features and methodology.

3 Overview of the technical approach

3.1 Method overview and encoding example

Many recent approaches to session-aware recommendation and to hybrid session-based recommendation propose entirely neural architectures, as discussed in Sect. 2. However, in particular when it comes to modeling long-term preferences and to combining them with short-term user intents, we found that such neural models often struggle to effectively leverage the available information (Latifi et al. 2021). In this work, we therefore propose an alternative approach, which (i) leverages the power of recent (neural and non-neural) methods for *session-based* recommendation and (ii) combines the output of these models with long-term preference signals and side information which we encode in a uniform manner as tabular data.

A particular novelty of our tabular data representation lies in the way we encode temporal (sequential) information enriched with a huge set of features and augmented with predictions from session-based recommenders. Besides the design and incorporation of novel model features, we also demonstrate that our hybrid ensembling framework yields higher increases in accuracy than often observed with newly proposed methods. Our finding suggests that different measures such as enriching various existing approaches with an extensive set of features capturing additional information can be more effective to improve performance of session-aware recommenders. This is an important finding for future work and for practitioners.

We illustrate our general approach with the following example. In session-based (and session-aware) recommendation settings, we are given a sequentially ordered log of recorded user interactions (Quadrana et al. 2018) as an input. Let us assume that the last session *s1* by user *u1* in the log looks as follows in simplified form:

$$s1(u1) : V1, A2, V4, A3, P3, V12, V7, A7, P7$$

where *V1* means a recorded *item view* event for item *i1*, *A2* refers to an *add-to-cart* event recorded for *i2*, *P3* is a *purchase* event for *i3* and so on. Table 1 shows a tabular encoding of this information, extended with side information and engineered features. The first group of columns represent the information about the interactions. The second group contains possible examples of side information about items (e.g., category and prices), and the last group of columns shows an example of engineered features that may also relate to long-term preference information. Here, *nb-prev-int* might express how often user *u1* has previously interacted with item *i1* in earlier sessions.

The data in Table 1 so far represent the observed interactions. To be able to learn sequential patterns in this tabular representation, we further (virtually)[3] augment the training data as follows. For each row in the table, we generate additional rows that represent events in the session that happened after it. Also, the tabular representation so far only encodes the actual observations from the log data, i.e., when a user has interacted with an item later in a session. For effective training, we however also augment further *negative* observations in our method, items the user did not view.

[3] We actually do not generate all these rows as will be discussed later.

Table 1 Possible tabular encoding of session events. The bold column shows the original sequence of item interactions from the example

SessionID	*Time*	*User*	***Item***	*Event*	*Category*	*Price*	…	*nb-prev-int*	…
s1	t0	u1	**i1**	View	C1	150	…	3	…
s1	t1	u1	**i2**	Add-to-cart	C2	170	…	2	…
s1	t2	u1	**i4**	View	C1	110	…	0	…
…	…	…	**…**	…	…	…	…	…	…
s1	t7	u1	**i7**	Purchase	C3	50	…	4	…

Enriching the training data with all possible items that the user has not interacted with would usually be too expensive, and selecting a random subset of non-interacted items might not be too informative. In our method, we therefore apply a novel approach for data augmentation in this respect. Specifically, we take the first l recommendations generated by a set of session-based techniques[4], which are trained in a rolling manner, at a given state of the session, enrich them with many additional features as outlined below, and correspondingly create additional rows (as negative examples) for the training dataset. This way, the algorithm can also learn in which cases a baseline recommendation strategy (e.g., that uses items from similar sessions) fails.

At this stage, we are now able to create the actual data in the format that is used for learning. In our present work we focus mainly on learning the most significant positive events in the data—which are *add-to-cart* and *purchase* events—, and we collapse these actions into a *target* label, which can be true or false. However, in addition we also consider the case where *views* are the positive examples, which is the common approach in the literature.

The final data in the form that will be used for model training by our second phase method will look like the example in Table 2. Before we go into details on the reasoning of how to generate the final form of data and specific examples, we want to provide a high-level overview to make our approach clearer. Specifically,

- We consider certain points in time for each session and all additional features are calculated with respect to the corresponding times. E.g., when the "current" time is 10 min after the start of the session, various features measure the user behavior and other aspects in these 10 min of the current session as well as the sessions before. Note that for model training, the term "current" refers to the considered point in time in the past.
- Based on the considered "current" times from the previous bullet point, the task is to predict what a user will do in the remainder of these sessions, after these times.
- Many potential "future items" are considered, which each user might interact with in the remainder of the session, after the considered point in time.
- The *positive* examples of future items are the actually viewed items for the view prediction problem and the actual add-to-cart and purchase events for the add-to-cart/purchase prediction problem. However, only those that were selected by the first phase session-based baseline recommenders as potentially relevant items are

[4] In our experiments, we used two different session-based baseline approaches, *GRU4Rec* and *S-SKNN*.

Table 2 Augmented tabular encoding with one row for each future candidate item to be predicted, for a given time $t2$ of session $s1$. The bold column shows examples of potentially relevant future items

S-ID	*Time*	**FutItem**	*FutEvent*	*Target*	*CurrCat*	*FutCat*	*SameCat*	*PriceDiff*	…
s1	t2	**i12**	View	0	C1	C1	True	− 10	…
s1	t2	**i3**	Purchase	1	C1	C1	True	20	…
s1	t2	**i10**	Non-view	0	C1	C3	False	50	…

retained (i.e., being among the top l predicted items, as described above). This is done because in production we also only have such predictions available.

– The *negative* examples of future items are the ones predicted by the first phase session-based baseline recommenders which were not viewed afterwards. Additionally, for the main add-to-cart/purchase prediction problem also the *actual* items that were only viewed but not added or purchased afterwards are used as negative examples for the training.
– This means that for every such considered current point in time, we have one additional training data row for every positive and negative example based on events that happened after that point in time. And we have many additional columns that represent various features, including the first phase recommender predictions for view or add-to-cart/purchase relevance. The features related to the items and their attributes are mostly calculated with respect to the potentially relevant future items, which might not be present in the respective users' history.

Now, we hightlight the aforementioned points by means of an example and provide more details on the reasoning. Assume we are at $t2$ in session $s1$ from the example. I.e., the current interaction is a view of item $i4$. Based on the observations seen so far ($V1$, $A2$, $V4$), a session-based recommender returns the following top-l items as recommendations: $i12$, $i3$, $i10$.[5] These items from the session-based recommender (or several of those) form the set of future items to be scored by our second phase GBM approach. I.e., on the one hand, a filter is applied in the end so that only those items are retained as positive and negative examples which are included in a baseline recommender. On the other hand, all items generated by the session-based baseline recommendation approach which were *not* observed in the remainder of the session are added as additional negative examples, labeled "Non-views". In our example, this means that $i10$ is added as additional negative example, since it is included in the baseline recommendations, but without any interaction in the remainder of the session. On the other hand, $i7$ is discarded for model building, even though there was an actual purchase event later in the session. This is done because we have no baseline rank in such cases, therefore the model could simply learn that a missing rank results in a purchase. And in production we also have only those items to score which a baseline recommender (or any other strategy for preliminary item selection) generated. We also tested an imputation of the rank, by using default values for items

[5] We assume that the items are given in this ranking order by the baseline recommender (we restrict to just one baseline recommender and a few items for illustration purposes). The ranks of the baseline recommenders are in fact provided as additional feature columns, which we omitted for space reasons in Table 2.

not included in the baseline recommenders for training. However, this did not result in increased performance. A possible explanation is the fact that in this case the model is trained with additional data not present in the validation and test sets. Thus, patterns are learned which are no longer present after training, since this information can not be supplied in production.

New features are then generated in a similar form to the ones in Table 1, but with respect to the *future* items to be predicted. This is illustrated in Table 2. The column "*FutEvent*" is the base for generating the target. I.e., for future events which are views or non-views the target is 0, otherwise it is 1 (of course, *FutEvent* is like the *Target* not included as a feature). The column "*CurrCat*" is the currently viewed category. It has therefore the same value in all rows for time $t2$ for the given user $u1$ in session $s1$ and corresponds to the category $C1$ of the viewed item $i4$ at $t2$, as visible in Table 1. The column "*FutCat*" corresponds to the category of the future item candidates for a recommendation. Therefore, the values depend on item attributes which might not be present in user $u1$'s history. From item metadata (not depicted), we obtain the category values $C1$ for items $i12$ and $i3$, and $C3$ for item $i10$. One relevant feature can be "*SameCat*" which simply checks whether the category of the candidate future item to be scored is the same as the currently browsed category. In this case, a purchase seems to be more likely, as a category interaction often reflects the current interest. Another important feature can be "*PriceDiff*", which measures the difference between the price of the currently viewed item and the price of the recommendation candidates. I.e., it indicates how much cheaper or expensive a future recommendation candidate item is in comparison to the currently viewed one. In our example in Table 2, we subtract the price \$110 of item $i4$ with the prices \$120, \$90 and \$60 of items $i12$, $i3$ and $i10$, respectively. Note that the latter prices can not be read from the example tables. In fact, most recommendation candidates are items a user never interacted with. Their prices, which can change in general, have to be calculated from dynamic (time dependent) item metadata or if this is not available, inferred from the most recent observations of other users with the same item in the log data.

To summarize, in the example in Table 2 we see that while being cheaper, item $i10$ is from a different category than the user is browsing and thus maybe less relevant to his or her current interest. The items $i12$ and $i3$ on the other hand are from the same category as the currently viewed item. And despite the fact that the baseline recommender judged that $i12$ has a higher ranking than $i3$, item $i3$ is (currently) cheaper. Therefore, with this additional information, our GBM model trained on this data can learn that actually $i3$ has a higher probability of purchase as a recommended item, because it is a cheaper substitute from the same category and has a resulting purchase as shown in Table 2.

Note that during training, the columns *S-ID*, *User*, *FutEvent* and *Time* are not considered. In the data used in the experiments, *time* is an absolute time stamp, which is not too informative. Instead, we include different engineered features like the time since the start of the session or the number of observed interactions since the session started.

The features in our final model are mostly more complex than the previously described ones. But we already saw that we can not simply make use of the interaction logs directly (as in Table 1, which itself already includes features we engineered). For instance, for time-based features we have to consider time deltas between features engineered with respect to the original log data and the future items. In Sect. 3.2 we provide more details of our algorithmic approach including the feature engineering based on an example.

3.2 Algorithm outline

In the following, we will outline our algorithmic approach in Algorithm 1 with the steps starting from preprocessing to model building and generating the outputs. Here, we limit our descriptions to a specific example of one additional engineered feature related to the price sensitivity of a user compared to other users, to have a more concise representation. For other features, different steps can be required, such as a logic for a time-based mapping, customized aggregations etc. For example, when calculating the time since the last and first interaction with a target category by a user, one can not directly use Table 1. In fact, there might have never been any past interaction with a category of a new item to be recommended to a user and thus no entries in Table 1. Therefore, we need to check whether there was a past interaction with the new category in Table 2 for this user in Table 1. And if so, both tables are needed to calculate the required time statistics, since this has to be done with respect to the current times in Table 2. The computational principle of combining Tables 1 and 2 with the respective constraint is shown in Line 19 of Algorithm 1. Note that DT is an abbreviation for "data table" in the algorithm.[6]

Algorithm 1 generates as a partial result in a rolling manner for every point in time the most recent price sensitivity with respect to the future category of the future candidate item for which a prediction shall be made (all the other features are omitted in this example). It also ensures that there is no data leakage. While it would be easier to calculate the price sensitivity over the whole training set, it would imply that future training observations are included, which are not yet known at earlier points in time in the training set.

Using a moving time window approach with such time buckets is based on the reasoning of providing the most up to date estimates by the baseline recommenders in the training set, as this will be also the case for production. The time-based split into training, validation and test was motivated by mimicking the situation in production, where we train a model on past data beforehand and cache its results. When we have this model, we in turn deploy and evaluate it on future data. The final output of Algorithm 1 is a list of recommendations sorted by their estimated relevance for purchase for every session and time of interest.

We want to emphasize that while we gave a specific example for one generated feature for illustration purposes, Algorithm 1 is a generic approach. The same principle can be used for other features. Depending on their nature, different constructions might

[6] Note that this eventually results in a big table including all features in similar form to Table 2 from the previous example.

Algorithm 1: Outline of the proposed algorithmic approach with user price sensitivity as an example feature. The approach is generic and other features can be added with the same principle.

1 Input: A preprocessed session–user–item–interaction records table *SessionItemTimeDT* (like Table 1).
2 Add category, price and other information (like availability) by joining *SessionItemTimeDT* with an item metadata table (in a time-based manner, since some attributes are dynamic and change over time). Ensure a session, time-based ordering of *SessionItemTimeDT*.
3 Split the data into k time-based buckets $B_i := [t_i, t_{i+1})$.
4 **for** *every session-based baseline recommender BaseRec* **do**
5 **for** *every time bucket index* $i \in \{0, ..., k-2\}$ **do**
6 Train *BaseRec* on the session-log data from *SessionItemTimeDT* with the time restricted to $Timestamp \in [t_0, t_{i+1})$, yielding a *BaseRecModel*.
7 With the trained model *BaseRecModel* generate session-based predictions and their respective rankings for all data in *SessionItemTimeDT* with $Timestamp \in B_{i+1}$.
8 **end**
9 **end**
10 Merge the results of the previous step **4** with *SessionItemTimeDT* in order to obtain a table *SessionFutureItemTimeDT*, storing for every session (with corresponding user) and selected time stamps a set of top l future candidate items from the session-based baseline recommenders and their respective rankings.
11 Filter *SessionItemTimeDT* on add-to-cart and purchase events and calculate with grouping operations:
12 (i) The cumulative mean of the item prices aggregated just by CategoryID (over all users).
13 (ii) The cumulative mean of the item prices aggregated by CategoryID and UserID.
14 Create a new column "*Relative price sensitivity of the customers for add-to-carts and purchases*" based on the difference:

$$(iii) = (i) - (ii) \tag{1}$$

15 For the new field *(iii)* of *SessionItemTimeDT*, carry the last observation forward, to have a value of the most recent relative price sensitivity for add-to-carts and purchases. This is needed to have values for view events too (otherwise they would be undefined, as they are calculated on the subset of add-to-cart and purchase events).
16 Aggregate by UserID, CategoryID and Timestamp to obtain a table including the latest relative category dependent price sensitivity for every user, named *AggUserCatTimeDT*.
17 By aggregating the result table *UserFutureItemTimeDT* from step **10**, construct a table with SessionIDs, Timestamps, future candidate items (FutureItemIDs) and respectively calculated future categories (FutureCategoryIDs) to be scored, obtained by joining the future item candidates from the baseline recommenders with metadata for those items. This table is named *AggUserFutCatTimeDT*.

be needed in the middle part (like from the filtering in Line 11 to the creation of new fields in Line 16) of Algorithm 1. E.g., different construction approaches need to be performed for moving window aggregations or other relevant statistics. Moreover, different entity-combinations are involved (e.g., some features depend on a user–item level instead of a category level). But we provide templates covering many cases in which for example different entities can be specified and similar features are derived automatically, making this process easier.

18 Perform a rolling non-equality join between the result tables **16** and **17** under the conditions that:
19

$$\text{UserID}(AggUserCatTimeDT) = \text{UserID}(AggUserFutCatTimeDT) \quad (2)$$

$$\text{CategoryID}(AggUserCatTimeDT) = \text{FutureCategoryID}(AggUserFutCatTimeDT) \quad (3)$$

$$\text{Timestamp}(AggUserFutCatTimeDT) = \max(\text{Timestamp}(AggUserCatTimeDT)), \quad (4)$$

$$\underline{\textit{such that}}\ \text{Timestamp}(AggUserCatTimeDT) \leq \text{Timestamp}(AggUserFutCatTimeDT). \quad (5)$$

20 We add many other features to the resulting table of the previous step, generate other tables for different entities like the user and the future item, and merge them all together to the table *SessionFutureItemTimeDT*, which also contains the predicted ranks by the baseline recommenders as features.[6] After adding the targets, we call this table *FeaturesTargetDT*. It is of the same form like the example from Table 2, with more features as columns and all potentially relevant items—for which add_to_cart/purchase or view probability predictions are made—as additional rows.
21 Split the data in *FeaturesTargetDT* in a time-based way into training, validation and test sets.
22 **for** *every* $s \in \{1, ..., n_{GBM}\}$ **do**
23 Train a GBM model GBM_s on the training data *TrainFeaturesTargetDT*, with a different seed than in previous steps, resulting in a different selection of random subsets of training data examples and random subsets of features.
24 Compute the vector of predicted probabilities $\widehat{y}^{(d)}_{GBM_s}$ based on the trained model GBM_s for $d \in \{train, validation, test\}$.
25 **end**
26 Compute $\widehat{y}^{(d)}_{GBM} = f_{avg}\left(\left\{\widehat{y}^{(d)}_{GBM_s}, s \in \{1, ..., n_{GBM}\}\right\}\right)$, ensembling the element-wise probabilities to obtain final predicted probabilities. We tested both the mean and median as functions to combine the predictions of individual models.
27 Join the results given by $\widehat{y}^{(d)}_{GBM}$ with the IDs of *FeaturesTargetDT* and group by SessionID, Timestamp to calculate a ranking based on predicted probabilities for each FutureItemID. These rankings determine the order of the output recommendation list (a sorted list of FutureItemIDs) for the given time stamp of a session.

4 Details of the two-phase approach and feature engineering

After having presented an overview of our overall algorithmic method in Sect. 3.2, in the following we provide the technical details of both phases including the feature engineering approach.

4.1 First phase recommender methods

The main reason for the use of the first phase recommendation algorithms is the fact that effective machine learning algorithms for tabular data—such as GBMs—have no natural mechanism to select relevant items not yet seen by a user. Therefore, there is a need to deal with this bias and the sparsity arising from the fact that users are unaware of the existence of a large fraction of the available items. This is a task for which recommender methods excel. As mentioned in the related work section, we use both the neural *GRU4Rec* (Hidasi et al. 2016a) method and the nearest-neighbor method *S-SKNN* from Ludewig and Jannach (2018) in our experiments to determine potentially relevant items.

The work of Hidasi et al. (2016a) introduced recurrent neural networks, in particular gated recurrent units (GRUs) to the next view prediction problem for session-based recommendation. The motivation is the fact that RNNs are tailored to model sequences and session-based recommenders deal with such sequences of view data. The difference of *GRU4Rec* to usual GRUs is mostly given by the different objective, which is a pairwise ranking based loss function. Moreover, the specific form of the batchwise training input data differs, in particular session parallel mini-batches are used.

The Sequential Session-based kNN (*S-SKNN*) method (also known as SeqContextKNN) from Ludewig and Jannach (2018) is inspired by the successful traditional nearest neighbor methods, adapted to a session-based context. In particular, instead of finding similar users or similar items as in user–user or item–item Collaborative Filtering approach, similar sessions are determined so that items from similar sessions can be used as recommendations. Technically, it is a sequence-aware method that puts more weight on recent observations, and is based on cosine similarity as well as in-memory index data structures for faster computation.

The intuition for combining both *S-SKNN* and *GRU4Rec* as preliminary item selectors is that they rely on different principles and thus learn different patterns in the data. This was partially confirmed in our experiments, although the *S-SKNN* method overall worked notably better for our datasets.

4.2 Second phase GBM algorithm

The idea of our proposed second phase approach is to leverage the predictions from one or several state-of-the art first phase session-based or session-aware recommender methods in a much richer context. Specifically, we propose a generic and extensive feature engineering approach (with specific examples provided in the next Sect. 4.3) that enriches the predictions by the individual first phase methods. Given these predictions or predicted ranks and this set of additional features, we propose to use a machine learning method that is, for example, able to learn in which situations what kind of first phase recommender algorithm works better, based on all additional features. The efficacy of this approach is likely due to the fact that the first phase recommenders learn different patterns in the data, which can be synergistic, and the diverse set of additional features allows the second phase model to learn additional patterns, thus improving recommendation accuracy.

In particular, we use GBMs as second phase models. GBMs are state-of-the-art ensemble tree methods that can achieve a high prediction accuracy for tabular data with many features. They rely on the principle of gradient boosting. It is an iterative approach in which in every training step a new decision tree is added to the model by fitting a tree to the negative gradient of the loss based on the model built so far, thus minimizing the generalized residuals, which implies that there is a focus on the cases with the highest errors of the previous steps (Hastie et al. 2009). They work mostly by minimizing the bias from a bias–variance trade–off perspective, but common implementations also use ideas for reducing variance by subsampling training data points and features, similar to the bagging in Random Forests. Altogether, GBMs offer a high generalization ability and effective measures against overfitting. Since

they can learn a large weighted set of rules (based on the individual decision trees added by boosting) they seem to be tailored to our problem, since as we have seen in the examples in Sect. 3, user decisions can intuitively be described as such a set of rules. We used the LightGBM library (Ke et al. 2017) in our experiments, which offers a very fast as well as accurate and flexible implementation of GBMs.

4.3 Feature engineering for the second phase GBM method

As outlined in Sect. 3, the result of the recommender selection algorithms from Sect. 4.1 form the basis for our *HySAR* method making use of GBMs as described in Sect. 4.2.

Having for every session and time stamp in consideration a set of potentially relevant future items, we obtain data in the form of Table 2. Now the task is to enrich all these data points with features that are likely relevant for predicting which items the user will buy or put in his or her shopping cart. As demonstrated by the examples and Algorithm 1 in Sect. 3, this step involves several advanced data preprocessing techniques, like aggregations on different entities with customized aggregation functions as well as time-based rolling, non-equi joins. The latter terminology means not to merge on equality conditions, but for example to take the last existent time stamp in one table before the reference time stamp in another table, similar to the example provided in Algorithm 1.

Note that the model will predict the add-to-cart or purchase probability for all l items from the first phase recommenders. After sorting decreasingly by this probability, we obtain a ranked list of items that can be recommended to the user in the given session. These can be considered as the most probable candidates for being purchased after a recommendation.

In the following, we provide a list of the feature groups that we engineered this way along with motivations for their choice. The detailed list of all features can be found in Appendix A.

User–item–session specific features[7] are based on derived user–item information, restricted to a given session. This is done because a current session might differ substantially from previous sessions of the same user. E.g., on one occasion the user was looking for a new washing machine, while in another session the user intends to purchase consumables. Such session-specific features are often highly valuable additions to pure session-based approaches. The reason is that session-based approaches typically look at the sequence of items in a given session, but neglect further information like the current user behavior related to time, interaction types and prices. For example, in this group we included features capturing how high in rank each first phase recommendation method predicts a given item to be relevant in comparison to the other items. Together with the other features, our method can learn in which situations which first phase recommender performs better. We also included prior observation (i.e., viewing) times, which could be more predictive than the so called *dwell time*, since times are summed up in case a user switches between items. Another example

[7] Note that the items in this case do not refer to past or currently browsed items, but they correspond to all potentially relevant future items selected by the first phase recommenders, to be predicted for the user in the current session.

are absolute and relative price differences between the target item and the last item the user interacted with. Such information can be combined by the model learning with features that assess whether a customer is price sensitive according to his or her past buying behavior, as described below.

User–item specific features aim to capture the preference of a given user for every item for which a prediction is made, by taking into account the whole history of the user. These features mostly capture the user behavior related to the items to be scored by looking at the recency and frequency of his or her interaction as well as the type of interaction for every item in question. This is achieved by not only looking at the users' current session, as session-based approaches do, but also information about the users' past. Examples include the calculation of the number of interactions and the number of sessions since the last interaction. In addition to time-based features, the number of sessions since the last interaction provides further information, as very active users might have viewed an item rather recently but are less likely to be interested in it if they had many sessions after the last view. We also measured the prior observation time of an item by a user across all sessions. This can be particularly relevant for higher priced products for which the user spends a longer time reading the details across several sessions before making a decision. Also, price changes across sessions are taken into account. For example, if the price gets cheaper compared to the last time a user checked but not yet bought an item, a purchase is more likely. These features can be synergistic. E.g., a user might have spent a lot of time checking the item, but was hesitating only because of the price.

User–category specific features were created based on the motivation to deal with the fact that observations on the item level can be scarce, due to the long-tail problem of rarely sold or viewed items. This is why it is reasonable to incorporate some analogous features on the level of product categories, for which more observations exist. We furthermore used some price sensitivity features defined as differences between mean prices of items in a category and mean prices of items a user selected from this category. Moreover, we distinguish between views and purchases, since a user might for example end up buying products cheaper than the ones he or she viewed.

Session specific features were added with the reasoning that some attributes only relate to the given session (and implicitly the corresponding user, but neglecting his or her past and the specific items in the session) are also relevant predictors, like the duration of a session and how many items where already viewed or purchased. This information helps to decide whether future purchases or views in the current session are likely. Feature examples include the number of observed actions in a session and its duration, allowing for a distinction how much time a user spends to check items compared to clicking through new items. Also, cumulative numbers of different actions are calculated, since if—for example—a user put 5 out of 10 items he/she viewed last into the cart, the user is probably more likely to do so with other items as well. Moreover, the weekday and time of the day of a session reflect the fact that typically more purchases are observed on certain weekdays and hours of a day. Such features allow us capture such a periodicity.

User specific features were included since it is also plausible that the general user behavior over time is predictive for the users' next actions. For example, if he or she is a very active user with many sessions and a high number of purchases in the last 100

interactions, it is more likely he or she will make a purchase. On the other hand, this is less likely if the user only viewed items without buying anything before. Other examples include the cumulative number of unique items and categories, which measure the diversity of interests of a given user. Cumulative counts for different event types were also calculated by weekday, since this information can help the purchase probability prediction (e.g., an item which would usually have a high purchase probability eventually has a lower one on a day where a user typically does not buy anything). Hour of day statistics account for the fact that users can have different times for browsing and buying. E.g., viewing items during the day, but making the purchase later in the evening. The same applies to the day of the week statistics for every user. Note that like for the other groups of features, there could be more interaction types than view, add-to-cart and purchase events. But for our datasets, no additional types of events were available.

Product category specific features aim to determine the overall relevance and popularity of a given category, across all users. This includes price and sales statistics to judge the overall contribution of a category to the business figures of a company. Some price-based features are also used to relate them to the price of particular items, as described in the item specific feature group. Note that the target categories are likewise given by the items for which predictions are made, not the currently browsed categories.

Item specific features were additionally incorporated in our model. Because aside from other features related to item attribute metadata that were already part of the other feature groups we discussed so far, it is also meaningful to include features that only depend on items. Examples include their time-based statistics, such as purchases by all users for different time window lengths. These features can be a more reliable substitute to the cumulative count-based features as they capture item popularity for varying recent time frames, which can be strong predictors for future interactions. Other examples include the time since first and last item appearances, since they can furthermore roughly capture when an item became available or when it was removed from the listing or became less lucrative. The number of unique users who interacted with items model the fact that some items are often revisited by the same users, but are not as relevant for others. Price differences are also part of this group and for example capture whether an item is a higher priced brand product compared to the other items from the same category, which can be predictive in conjunction with the respective attributes inferred from user behavior.

Sparse categorical features are used to identify the same objects or groups throughout the whole dataset. This can help the generalization ability, as the model can implicitly learn biases (like a bias for very active users or very popular items, although such characteristics can be partially captured by other features). However, some of those categorical features in contrast led to overfitting, as did some other features, which is why they were removed in the final model. A possible explanation for this is the relatively high number of different IDs for some features. An alternative to this approach is mentioned in Sect. 8 on future works.

We iterate that the aforementioned feature groups are *examples* that we used in our experiments. However, the data that those features are based on are often present in e-commerce applications. Therefore, the same features that we provide in our templates

can be used in other e-commerce applications, which is a broad domain. Moreover, some of the features are also relevant in other application scenarios. E.g., on a music streaming platform the items will be songs, categories will be the genre and instead of a purchase, we can consider a strong indication of preference if a user listened a song to the end or multiple times, while viewing events are just shorter samples. This way, many of the features can in principle be reused in such a context. Moreover, as we described in Sect. 3.2, in our generic approach new domain specific features can be easily integrated.

5 Experimental evaluation

In this section, we present the setup and the results of our experimental evaluation.

5.1 Experimental setup

In the following, we describe the datasets that were used in the experiments, the preprocessing we applied, and the metrics that we use to assess the effectiveness of our approach.

5.1.1 Datasets, preprocessing and memory considerations

We used two real-world datasets for our experimental evaluation, both based on e-commerce data. The first one is the *Retailrocket* dataset, consisting of website interaction logs distinguishing between item view, add-to-cart and purchase events. The second dataset is called *Diginetica* and consists of item views and purchases of users and search queries, which are however anonymized. Both datasets provide additional dynamic item metadata with varying attribute values over time. The *Retailrocket* dataset consists of 2,756,101 logged interactions (events), while *Diginetica* contains 1,235,380 interactions. However, only 372,991 interactions of the *Diginetica* dataset have an associated user ID, which is required to make use of all possible features provided by our method. Unlike the *Retailrocket* dataset, the *Diginetica* dataset also does not contain add-to-cart but only purchase events. The *Retailrocket* dataset does not provide session IDs, but only user IDs. Therefore, new sessions were identified by idle times of more than 30 min, as often done in the literature.

We further applied some essential filtering on both datasets as it is common for experiments with recommender systems, including session-based recommenders. Specifically, we filter out items for which we observed less than 5 interactions, to focus on items that appear more frequently in different sessions, therefore strengthening learning by finding similar sessions. And naturally, we can only make use of sessions with at least two items. After preprocessing, we obtain the statistics shown in Table 3.

As outlined before in Sect. 3, our approach results in a high number of data points and can therefore have a high working memory demand, because every candidate item to be scored results in an additional data row (i.e., with l candidate items, there are $l \cdot$

Table 3 Statistics of the datasets *after* common preprocessing

Statistic/dataset	*Retailrocket*	*Diginetica*
Number of unique sessions	352,702	56,836
Number of unique items	79,777	18,550
Number of unique users	290,110	54,029

n_{feat} values for every observation in the original data).[8] Therefore, for the *Retailrocket* dataset, we used more recent data for training our proposed method *HySAR* for lower working memory requirements and faster computations. Specifically, we chose about 37% of the data, consisting of the more recent time buckets. Using older historical data in addition will likely further increase performance. For the same reason, we sampled one time stamp per session, but all sessions of each dataset are considered. For the *Diginetica* dataset, we tested using more than one time stamp per session, which resulted in slightly better results for our approach. But the difference is modest, suggesting that the amount of training data is big enough already, and the various prediction situations in a session (e.g., rather in the beginning or the end of a session) are covered by the high number of sessions.

5.1.2 Baselines, evaluation setup and hyperparameters

As discussed earlier, we chose the *S-SKNN* and *GRU4Rec* methods both as first phase recommenders and as baselines due to their high performance for e-commerce data as shown in Ludewig and Jannach (2018). As in the original papers, predictions are generated in a "rolling" manner for the next item. I.e., only one step ahead is forecasted instead of the entire remaining session. This approach is also used to generate data for our *HySAR* method (every predicted candidate item yields a row, cf. Sect. 3.1).

We both report the accuracy with respect to an evaluation scheme in which the ground truth are the immediate next items and in which all following items are considered as the ground truth. We discuss this in Sect. 5.3. Note that even in case of the next item evaluation, there can be cases of several next items with the same time stamps in the data (e.g., simultaneous purchases). In this case, both are included in the ground truth.

We used a temporal splitting based on the generated time buckets for the evaluation, with the test set being after the validation set and the validation set after the training set (with the exception of overlapping sessions). This way, we also have sessions of new users in the validation and test sets, as this can be also the case in production.

As described before, we focus primarily on the task of predicting item add-to-cart and purchase events, as these are the most important events in this application domain. We however also consider the "next-view" prediction problem that is commonly con-

[8] Note that a possible way to lower the working memory demand is to process the data batch-wise and store it on disk. E.g., when creating features depending on users and their past behavior, batches of users can be processed and stored. The same applies to different entities. The LightGBM package (Ke et al. 2017), which we used, also supports training with most data being on disk. Despite the memory demand, the running time for the feature generation and training of our *HySAR* method itself is typically lower than generating the predictions from the first phase baseline methods for all time buckets.

sidered in the literature. In the results of the evaluation shown in the next section, cases are excluded in which the future item (for which an add-to-cart or purchase action is predicted) coincides with the currently viewed item. This is appropriate, because in practice we would not want to recommend a currently viewed item to a user for purchase. This simple restriction however makes a significant difference in the absolute performance values. This is due to the fact that in many cases users click on an item and directly make a purchase afterwards.

In order to have a fair comparison, for the evaluation only those items were considered which were included in the first phase selection (based on *S-SKNN* and *GRU4Rec*, as described in Sect. 4.1). We observed that without that restriction, substantially higher accuracy values can be achieved. This suggests that using alternative first phase item selection strategies should be explored in future research.

In terms of the hyperparameters of our approach, we used $k = 20$ time buckets and we generated $l = 200$ candidate items (per selected time stamp of each session) based on the first phase recommenders *S-SKNN* and *GRU4Rec*, according to the description in Sect. 4.1. Even better results for our method may be possible by systematically tuning these hyperparameters. E.g., higher values for k and l could lead to better results, but they also increase the memory demand as they result in an increase of the data.

We initially tuned the remaining hyperparameters of our method and baselines by random search. However, we found a configuration for the baselines as well as our proposed method and its GBM parameters that works well for both datasets and prediction tasks. While small improvements can be achieved by additional parameter tunings, we used this configuration. We do this because the consistency in the performance suggest a higher robustness for new datasets and various tasks, and a reduction of time needed for hyperparameter search. Connected to this, it is important to note that unlike in common cases, the performance of our proposed method *HySAR* and the baselines are not independent. Our baselines are not only baselines in the common sense, but at the same time serve as first phase item selection and scoring algorithms and their results are incorporated in our method by the second phase GBM algorithm (see Sect. 4). Since our method is a stacking approach, any performance increases by better parameters of the baselines usually also increase the performance of our proposed method. This is because the better quality of the predictions and the selected items is a benefit to the stacking GBM model, which is build on top of them.

5.1.3 Running time considerations

In the following, we want to address aspects regarding the running time complexity of our proposed method. To summarize the most important points regarding the computational demand, the generation of the first phase session-based recommenders takes the highest proportion of the running time. Compared to using only one first phase recommender directly without our approach, there is a considerable overhead, but this process has to be done *only one time* with all the available historical data. As described in Sect. 3.2, this is done by iteratively training models on all previous time buckets and making predictions for the next time buckets. The training time depends on the

chosen number of time buckets k and lowering k will therefore reduce the running time and may still give good accuracy. The same applies to other settings.

By noting that the successive training times can be expressed as a geometric series, we can estimate the one time running time demand to be approximately by a factor $(k-1)/2$ higher than training a first phase recommender on the whole data, assuming a linear increase with the amount of data. There is also an additional need to generate predictions from the trained first phase recommender on every time bucket, which can be longer, but these predictions might be already available in practice, since they correspond to the calculations needed to deliver recommendations to customers (i.e., the successive training and prediction for next time buckets can arise naturally in practice and in this case there is no initial computational overhead).

After these potentially needed one time additional computations, the required running time can be substantially decreased, since only incremental updates are needed to train the newest models (with all data at once or weight updates just with new data) and make predictions for the last time folds for the use in production. The same applies to the feature generation, the running time of which can also be decreased respectively by only calculating the features for the newest data and using cached data for the features based on historical records.

We have assessed more thoroughly the *one time* running time of our method for the whole historical data. We considered the bigger *Retailrocket* dataset and the *next view prediction* case commonly used in the literature. We further point out that we did not employ any performance enhancements. The *GRU4Rec* method was running on a machine with 6 CPU cores, while substantial performance increases could be achieved by using a GPU, which partially also would be the case for our GBM based method. The same applies to using a possible multiprocessing for the *S-SKNN* method, which was running on a single core in our experiments, as well as many parts of the feature engineering, since performance optimization was not in our scope for this paper.

The total time needed to train and generate the predictions of the *GRU4Rec* method for all historical time buckets was about 24 h 23 min in our experiments. The corresponding time for the *S-SKNN* method was shorter with around 11 h 14 min. The time needed for the feature engineering from all the historical data and generating optional cache files for future reuse was approximately 2 h 18 min, and training the second level GBM model of our *HySAR* method itself including the postprocessing took about 1 h 42 min.

Further note that aside the remedies to improve running times discussed so far, upscaling is more easily possible and of lower cost, since the calculation for the whole historical data has to be performed only once if required at all. For example, it would be possible by using cloud computing with instances having more resources. Also note that for the use in production, the period of model retraining can be adapted and for a short time frame of new data, tangible differences in performance are unlikely. Therefore, already trained models can be used. Choosing computationally better performing first phase methods is also an effective remedy in this regard, since this directly impacts the running time of our proposed method. Increasing the learning rate would offer faster training with only small accuracy decreases for training the *HySAR* GBM model in particular. Moreover, if a substantial amount of more recent data is available

for training, the less important it will likely be for accuracy to use older historical data, so restricting to fewer recent time buckets would also be an effective measure.

5.1.4 Evaluation metrics

We report the results with respect to several well-known metrics, which are tailored to measure ranking accuracy (i.e., the higher, the better), in particular for the top items. For this purpose, we use the popular *Mean Average Precision (MAP)* metric. Precision measures how many items from the list of the items with the highest predicted values were actually relevant and the precision values at different positions are averaged in the *MAP* metric. We report this metric at the threshold of 10, since the topmost items are the most relevant ones. In addition to Precision, *Recall* measures how many of all the truly relevant (e.g., bought) items are included in the prediction list. Specifically, we include *Recall@1*, which coincides with the *HitRate* in case of only one true item. This metric was used extensively in the previous literature on session-based recommender systems. We also report the common *Mean Reciprocal Rank (MRR)* metric, which is given by the mean of $1/rank_i$ over all cases i, where $rank_i$ is the rank of the first correctly predicted item in list i (e.g., the position of the first item in the sorted prediction list that was actually bought). The metric puts special emphasis on the fact that the first relevant item should be placed as high as possible in the recommendation list. In addition, we report the often used ranking measure *normalized Discounted Cumulative Gain (nDCG)*. In the definition of the *nDCG* that we used, the list of the actual items afterwards in a session determine the ideal score, each with the same relevance. The list of the predicted items (determined by the first phase recommenders and the score by the respective algorithm) is matched with this list of true items, and according to the definition of the *DCG* each position is logarithmically downweighted depending how far the true items appear in the prediction list.

5.2 Experimental results for the next item prediction task

We present our results both with respect to the central purchase or add-to-cart recommendation prediction problems, as well as the case of the next view prediction problem that is more common in the literature, as outlined in Sect. 3. The ground truth is given by the immediate next items. For the purchase or add-to-cart case, the results are reported in Table 4 for the *Retailrocket* dataset and in Table 5 for the *Diginetica* dataset, each with respect to the average values of all metrics. For the case of views, the respective results can be found in Table 6 for the *Retailrocket* dataset and in Table 7 for the *Diginetica* dataset. In addition to the performance results of our *HySAR* method, the performance of the *S-SKNN* and *GRU4Rec* methods themselves (see Sect. 4.1) are reported as baselines.

As we can see, our method consistently performs better than the baselines in a significant way. This can mostly be attributed to our new approach which involves the extensive engineering of additional predictive features used in conjunction with the first phase recommender predictions, leading to superior forecasting accuracy in our GBM models. We can furthermore observe that the *S-SKNN* method achieves a notably

Table 4 Results with the mean metric values for the purchase/add-to-cart prediction for the *Retailrocket* dataset, with the *next items* as the ground truth

Metric/results	*HySAR*	*S-SKNN*	*GRU4Rec*
Recall@1	0.1411	0.1030	0.0764
MAP@10	0.2140	0.1726	0.1199
MRR	0.2239	0.1804	0.1293
nDCG	0.2899	0.2427	0.2000

Table 5 Results with the mean metric values for the purchase/add-to-cart prediction for the *Diginetica* dataset, with the *next items* as the ground truth

Metric/Results	*HySAR*	*S-SKNN*	*GRU4Rec*
Recall@1	0.1640	0.0977	0.0536
MAP@10	0.2057	0.1452	0.0809
MRR	0.2287	0.1653	0.0960
nDCG	0.2571	0.2006	0.1393

Table 6 Results with the mean metric values for the view prediction for the *Retailrocket* dataset, with the *next items* as the ground truth

Metric/results	*HySAR*	*S-SKNN*	*GRU4Rec*
Recall@1	0.2024	0.1550	0.1219
MAP@10	0.2916	0.2335	0.1965
MRR	0.2995	0.2400	0.2059
nDCG	0.3781	0.3095	0.2913

Table 7 Results with the mean metric values for the view prediction for the *Diginetica* dataset, with the *next items* as the ground truth

Metric/results	*HySAR*	*S-SKNN*	*GRU4Rec*
Recall@1	0.1069	0.0894	0.0678
MAP@10	0.1935	0.1654	0.1301
MRR	0.2071	0.1765	0.1449
nDCG	0.3133	0.2663	0.2521

better result as a recommender than the *GRU4Rec* method. This might be surprising at first, but is consistent with previous findings in the literature for these datasets, that also showed lower performance results of *GRU4Rec* compared to *S-SKNN* for some datasets (cf. Ludewig and Jannach 2018 and Ludewig et al. 2021). This might be due to the fact that the assumption to estimate relevant items in a session by items of similar other sessions works well, also with respect to a limited amount of data, since for the smaller dataset the relative difference between the two baselines is higher. For other datasets (non-e-commerce or publicly not available ones) different results were found, with *GRU4Rec* sometimes performing better.

Despite the fact that *S-SKNN* works better, additionally including the predictions from the *GRU4Rec* algorithm does help the performance of our ensembling *HySAR* method. This could be explained by the fact that the types of recommendations differ between *GRU4Rec* and *S-SKNN*, as they take different approaches in learning from data. Note that for the *Retailrocket* dataset, higher absolute values can be achieved in

the case of a next view prediction than a next purchase/add-to-cart prediction, while these results are mixed for the *Diginetica* dataset. This can probably be attributed to the fact that some add-to-carts and purchases are performed later in a session, as confirmed by our experiments in Sect. 5.3. The absolute differences between both datasets are likely due to the different distributions and purchase patterns, as well as their respective sizes.

We further observe that the *GRU4Rec* method performs notably worse relatively to the other methods for the purchase/add-to-cart case compared to the case of views. This is especially the case for the most important top positions, while for higher thresholds the relative difference seems to decrease, suggesting that *GRU4Rec* does find relevant items also for the purchase prediction, but notably slower in this case.

Especially for the *Diginetica* dataset, the purchase prediction results in a higher relative performance gain of our method over the best baseline compared to the view prediction problem, while the gains are similar for the *Retailrocket* dataset. The comparatively better results of the *GRU4Rec* method in case of the view prediction compared to the purchase prediction likely improved the performance of our method for the view prediction in comparison to the purchase prediction. Another first phase method that works better for the purchase prediction would likely further increase the performance of our proposed method. Thus an overall higher gain should be possible for the purchase prediction, which is reasonable, since more features are available in this case (e.g., features distinguishing between interaction types, as described in Sect. 4.3).

It may also be possible to improve the case of the view prediction with more features based on purchase and add-to-cart events, too (e.g., as a simple example, viewing can become less likely after buying). However, the other approaches in the literature did not take into account information about purchases, so we also focused on views only to have a better comparison.

We furthermore note that for some experiments with the *GRU4Rec* method (in both the next and all remaining prediction case in the next section), earlier experiments led to somewhat better results than the final runs which were used for generating the first phase recommendations for our proposed method. We provided those values for the *GRU4Rec* method, which are about 10% (and up to 20%) higher, and a bit better results still can be possible for different runs. However, because of the gaps to the next best method, these differences do not change the overall result.

Additionally, we note that we conducted experiments in which we removed all features from our model that were based on the user and his or her behavior and we address these results in more detail in Sect. 7. We want to add that we also conducted preliminary experiments with additional features that are similar to the ones already mentioned. For example, it is possible to consider fractions of the existing running count based features by user comparing different event types (e.g., `run_count_add_to_cart/run_count_view`). Likewise, the same time window based features for the items can also be calculated on a user–item and user level by simply changing the aggregation IDs. And the time since the first interaction can also be used by user instead of only by the user–item interaction. The results indicate some improvements for the *Retailrocket* dataset, and modest gains for the *Diginetica*

dataset. We have not tested these variants for all cases however and therefore they are not included in the presented results.

5.3 Experimental results for an evaluation with all remaining items as the ground truth

So far, we considered the case in which the ground truth is based on the immediate next items, as it was often done in the literature. However, considering all remaining items in the given session as the ground truth set can be more reasonable, as argued in Ludewig and Jannach (2018). One reason is that a user might perform an action with a delay, after browsing other items. It also aids in addressing data sparsity issues. On the other hand however, preferences might change during a session and in such cases later items might not be as relevant as the items prior in a session, which is not taken into account if all remaining items are considered. We have also performed an evaluation with this extended set of ground truth items. However, although the evaluation was based on all remaining items, the model training was so far based on the next items only, as in the case of the next view or purchase prediction. For our proposed method, it is however easily possible to add later items from the remainder of any session to the training set. This is likely to improve the results, since training tasks and evaluation would be better aligned.

Moreover, there is a caveat in the case of the purchase/add-to-cart prediction. Since in our evaluation approach we so far randomly sampled a time stamp for any session, often a time stamp relatively early in a session will be used. These early actions might not be as representative for later purchase decisions, as argued above. Therefore, we used the time stamp of the last view before the first add-to-cart or purchase instead of a randomly sampled time in this case. Although this precise time is not known a priori in production, we argue that this assessment is important in practice, because many websites provide recommendations for supplementary items during the checkout right before the purchase. Moreover, a user might finally evaluate any substitute articles before making the purchase.

The results for the evaluation with all remaining items as the ground truth are shown in Tables 8 for the *Retailrocket* and in Table 9 for the *Diginetica* dataset for the case of add-to-cart/purchase prediction. For the view prediction, the results can be found in Table 10 and in Table 11, respectively.

We can observe from the results that our method still performs best and the overall ranking stays the same, but the relative performance gain of our method over the baselines is a bit lower compared to the next purchase or view prediction problem

Table 8 Results with the mean metric values for the purchase/add-to-cart prediction for the *Retailrocket* dataset, with *all remaining items* as the ground truth

Metric/results	*HySAR*	*S-SKNN*	*GRU4Rec*
Recall@1	0.1416	0.1159	0.0899
MAP@10	0.2197	0.1830	0.1419
MRR	0.2628	0.2209	0.1746
nDCG	0.2973	0.2526	0.2198

Table 9 Results with the mean metric values for the purchase/add-to-cart prediction for the *Diginetica* dataset, with *all remaining items* as the ground truth

Metric/results	*HySAR*	*S-SKNN*	*GRU4Rec*
Recall@1	0.2053	0.1551	0.0486
MAP@10	0.2687	0.2154	0.0951
MRR	0.3175	0.2546	0.1168
nDCG	0.3411	0.2906	0.1776

Table 10 Results with the mean metric values for the view prediction for the *Retailrocket* dataset, with *all remaining items* as the ground truth

Metric/results	*HySAR*	*S-SKNN*	*GRU4Rec*
Recall@1	0.1901	0.1475	0.1164
MAP@10	0.2865	0.2298	0.1935
MRR	0.3287	0.2663	0.2280
nDCG	0.3837	0.3138	0.2974

Table 11 Results with the mean metric values for the view prediction for the *Diginetica* dataset, with *all remaining items* as the ground truth

Metric/results	*HySAR*	*S-SKNN*	*GRU4Rec*
Recall@1	0.0974	0.0837	0.0610
MAP@10	0.1880	0.1613	0.1248
MRR	0.2601	0.2258	0.1865
nDCG	0.3305	0.2805	0.2674

from the previous section. We assume this to be the case due to some heterogeneity in the sessions with preference shifts in between, as well as the fact that our method was only trained with the next items rather than all remaining items in the session, as we described above.

Moreover, it is noteworthy that with exception of some cases for the *Recall@1* metric, the absolute values are higher for all methods compared to the next item prediction case, which can probably be attributed to a bigger set of ground truth items. The fact that this is not always the case for the *Recall@1* metric could be explained by the same reason, since many relevant items in this case are not considered at a threshold of 1.

Note that we also performed statistical significance tests for all cases (both datasets, prediction targets and next item as well as all remaining item predictions). In accordance with previous works from the literature, we used a Wilcoxon signed rank test at a level of $\alpha = 0.05$ to measure the significance in difference between our proposed method *HySAR* and the best performing baseline. We found that all performance differences are significant, with many p-values being considerably lower than 0.05. The comparatively higher p-values were observed for the add-to-cart/purchase prediction for the next item case and the Recall@1 metric, considering only one item. This is reasonable, since in these cases the ground truth sets are smaller.

6 Feature impact analysis

In this section, we investigate what features contribute to what extent to our overall GBM based recommendation prediction model. As mentioned in the beginning, we make use of the *SHAP* method (Lundberg and Lee 2017; Scott et al. 2020) with a corresponding shap library which is based on a game theoretic attribution concept using Shapley values. It allows for a fair assessment of the contributions of every feature to the overall model. Recently, many papers have been published on explainable AI, some of which address recommender systems. However, to our knowledge, none has performed an analysis similar to ours. The papers we found in the literature either used a different methodology, different types of recommendation (i.e., not session-aware) or a different application. See, e.g., Afchar et al. (2022) for explainability in music recommendation, or a movie recommendation context in Roberts et al. (2022). Some other papers also made use of the *SHAP* method, but with a different intention and not for e-commerce data. For example, it was used after a clustering of users for movie preference data, to determine which clusters users belong to (Misztal-Radecka and Indurkhya 2021). In Geng et al. (2022), a session-based recommendation setting is considered, but with the purpose of causality modeling. Many papers in this area also focus on explanation to the users, while our intention is to provide explanations to the companies and developers of recommender systems, in order to improve the latter and get more intuition into the user behavior.

We focus on our main purchase/add-to-cart prediction problem. The results are shown in Fig. 1, which includes the top 20 features of highest importance, ordered decreasingly by the importance value. We notice that many feature rely on the category rather than the item, which could be explained by data sparsity, with more observations being available on the category level. We want to emphasize that the features listed in this chart were calculated with respect to the target item and its category to be predicted, not the currently browsed item or category.

Every row in this chart represents a feature and the points in the corresponding row are data examples which are colored according to the value of the respective feature, based on the overall distribution. Red colors correspond to high values, while blue colors represent low ones. Gray corresponds to unknown values. Furthermore, the points are ordered on the horizontal axis based on their impact on the model output (SHAP). That means, the further a point is to the right, the higher is the increase in the output purchase probability because of the corresponding feature value. The opposite holds for points on the left, for which the respective feature value lowers the probability.

The following insights can be derived from Fig. 1:

– The top ranked feature (`future_item_rank_base_1`) in fact corresponds to the predicted rank by the first phase recommender *S-SKNN* in this case. Since low rank values correspond to a high score of an item by *S-SKNN*, the blue points are the ones the *S-SKNN* method estimates to be the most relevant items. And in these cases, the purchase probability generated by our *HySAR* method is increased as a result.

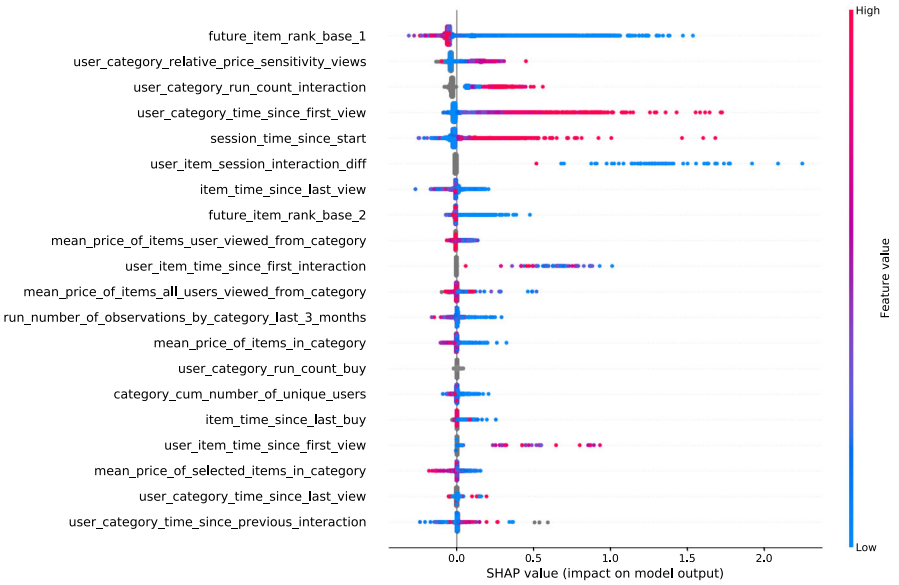

Fig. 1 Feature importance based on Shapley values. We show the top 20 most important features, sorted in descending order. Every point is a data example. The colors reflect whether the feature values are high (red) or low (blue). Gray colors correspond to NULL values of the corresponding feature. The further a point is to the right, the higher is the increase in purchase probability due to this feature value. On the other hand, for points on the left the probability gets lowered because of the respective feature value

- The predicted rankings by the *GRU4Rec* method have less impact (ranked number 8), but this feature is still among the top, showing that both first phase baseline recommenders can produce session-based recommendations useful for our second phase personalized session-based *HySAR* method.
- The feature with the second highest impact is price sensitivity related (`user_category_relative_price_sensitivity_views`), and is defined as the difference between the mean price of items from the target category viewed by all users and the mean price of items from this category the given user viewed. I.e., a collaborative effect is being taken into account by making use of the browsing patterns of all other users, leveraging all historical data. Interestingly, by analyzing the color distribution on the left and right hand side, we see that more price sensitive customers seem to have a higher willingness to buy when they find a good offer. This suggests a significant role of bargain hunters.
- The last finding matches with the pattern of another feature, namely the mean price of items of the respective category that the user has viewed: the lower the price, the higher the chances of purchase. Because the blue points representing low mean price values are concentrated on the right. However, there are some users who show an opposite pattern (red dots on the right). Note that our model can learn interactions between this feature, the previous price sensitivity feature and the item price itself in order to decide what items match to the price expectations of users.

- The feature with the third highest importance is the running (cumulative) count of interactions in the respective category of the future item by the given user. This feature does not only take into account views but also purchase events and is also calculated across all past sessions. This demonstrates that information prior to the current session can be valuable. Note that the gray dots correspond to new category interactions by a user, ones he or she did not interact with before.
- The next feature is the attribute describing the time since a user first viewed any item in the target category across past sessions, also capturing long-term interest. Indeed, as we can see on the right-hand side, higher times increase the purchase probability as they often correspond to loyal customers.
- On rank 5 we have the feature representing the time since the start of a session. It shows the intuitive fact that the longer a current session is, the more likely a user is going to buy some items.
- The next feature `user_item_session_interaction_diff` describes how many sessions ago a user last interacted with the target item for which a scoring is made, either by viewing or buying. This feature can be seen as capturing the recency with respect to the user—item interaction.
- The time since the last view of an item across all users on rank 7 indicates the current popularity of the item in question. E.g., when there is a new promotion on an item, it is viewed very frequently and the time is therefore very short.
- It is interesting to see that for `user_item_time_since_first_interaction` at rank 10, we have a mixture of both high and low values on the right-hand side, therefore increasing the purchase probability. This means that in some cases very recent new item interactions by users lead to a purchase. But on the other hand, in many cases also longer times increase the probability, depending on other features.
- The feature `run_number_of_observations_by_category_last_3_months` at rank 12 captures the recent popularity of the respective target category.
- There are also several other item based features placed a bit lower, such as the last purchase of an item across all users and the time a user first viewed an item across all his or her sessions. There are also features capturing the latest interactions by users, such as the second last on the list, recording the time since a category was last viewed by a user.

To summarize, the findings in this analysis are intuitive yet enlightening. An analysis based on Shapley feature importance values on top of our recommendation approach offers good interpretability and better business understanding of the customers in an organization. This does not only help customers in their decision making process, but also aids in managerial decisions for example in departments involved in customer relationship management.

7 Ablation study

A feature impact analysis with the *SHAP* method as outlined in the previous Sect. 6 can give a good intuition about what types of features are important. Also, some works use

Table 12 Ablation study results for the purchase/add-to-cart prediction for the *Retailrocket* dataset, with the next items as the ground truth

Variant/metric	Recall@1	MAP@10	MRR	nDCG
BasicSessionInformation	0.1096	0.1803	0.1907	0.2640
NoUserItemInteractionInformation	0.1201	0.2039	0.2145	0.2838
NoUserInformation	0.1168	0.2013	0.2118	0.2818
DefaultModel	0.1411	0.2140	0.2239	0.2899

Table 13 Ablation study results for the purchase/add-to-cart prediction for the *Diginetica* dataset, with the next items as the ground truth

Variant/metric	Recall@1	MAP@10	MRR	nDCG
BasicSessionInformation	0.1097	0.1627	0.1804	0.2230
NoUserItemInteractionInformation	0.1359	0.1788	0.2035	0.2373
NoUserInformation	0.1538	0.1908	0.2123	0.2458
DefaultModel	0.1640	0.2057	0.2287	0.2571

Shapley value based ablation in order to increase performance by removing a subset of features (e.g., Chen et al. 2022; Ukil et al. 2022). However, while the *SHAP* method as discussed in Sect. 6 has several desirable properties such as local accuracy and consistency as described in the respective references, feature importance contributions in general do not directly translate to differences in overall accuracy.

Thus, since there is no direct association between Shapley value based feature importances and global model performance, we also performed an ablation study, in which we tested a few variants of our proposed model with different sets of features selected or removed. We chose these sets as intuitive groups of features describing different types of side information to be considered or not, e.g., relating to whether or not user based or interaction based information is incorporated.

We consider the next purchase/add-to-cart case and compare the default model from Sect. 5.2 with different variants in which certain types of features are excluded. The results are summarized in Tables 12 and 13 for the *Diginetica* and the *Retailrocket* dataset, respectively.

The *BasicSessionInformation* variant considers the first phase recommender prediction alongside some basic session information (in particular the number of views and purchases in the given session, the time since its start and the observation time of an item by the user in the current session). As we can see from the results, the performance of this model variant is better than the best baseline, but notably below the values achievable by the *DefaultModel*, therefore highlighting the benefit of including additional features capturing various types of information.

The *NoUserItemInteractionInformation* variant is a model that does not include information related to the interaction of a user with a particular item or the corresponding item category, including in the given session. It however includes features like the number of unique items a user interacted with, which does not depend on a certain item. The results show that this model variant achieves results between the *DefaultModel* and the *BasicSessionInformation* model, demonstrating that both the

excluded user–item interaction features as well as the remaining features are important for the model accuracy.

The *NoUserInformation* variant corresponds to a pure session-based approach that neglects prior and side information about users. The variant however still comprises user based features that only depend on the given session (e.g., the observation time of an item by the given user in the current session). As we can see, there is also benefit in incorporating user based information and this variant also lies between the values of the *BasicSessionInformation* variant and the *DefaultModel*, being mostly closer to the *DefaultModel*. For one dataset, the values are close to the *NoUserItemInteractionInformation*, while for the other one the removal of user based features seems to matter less. The fact that this model variant is mostly close in performance to the default model highlights that short-term session-based information is most important and information about the user from previous sessions helps, but provides a comparably limited gain. This might also be due to the fact that in the used datasets, there are many users with just one session.

Overall, we can infer that different types of features contribute to the overall model performance and that it can be worthwhile to perform such analyses as the relevance of each feature group may differ depending on the data at hand. The results suggest that more types of features can improve the model quality if the corresponding information is available in practice.

8 Conclusions and future works

Existing research in the area of session-based and session-aware recommendation largely relies on pure collaborative filtering approaches, where recorded user–item interactions are the main or only basis for learning next-item prediction models. Only limited work exists so far in terms of leveraging various types of side information in the recommendation process, which are commonly available in practice. In this work, we have proposed a method that combines the power of existing session-based or session-aware models with extensive generic feature engineering techniques in a GBM-based ensembling approach. Our experimental results clearly indicate that substantial performance increases in session-based recommendations can be achieved by combining the predictions of several state-of-the-art session-based recommender methods with a second phase machine learning based approach that incorporates a high variety of features.

While our experiments so far are limited to the domain of e-commerce, it is important to note that the core of our method is generic and not tied to a particular application domain or certain types of item side information. Arbitrary static and dynamic attributes from a given application can be incorporated, and many of our current features are actually domain-independent, e.g., those relating to the number of previous interactions or time-related ones. Moreover, any other additional session-based or session-aware recommendation algorithm can be used in the first phase of the computations, further improving them.

There are several directions for future research. As mentioned above, it is important to perform additional evaluations in other applications domains. Such evaluations may

mainly require an additional feature engineering step for those features that are specific to the application.

From a technical perspective, the overall performance of our method depends on the quality of the item selection task in the first phase.[9] Apart from using alternative session-based or session-aware recommender algorithms, alternative ways of preliminary item selection can be explored. This may, for example, include advanced sampling techniques, augmenting with trending items, learning models on different hierarchies and to separately compute factorization models or embeddings in an iterative way in order to determine the most similar items to given ones in a session.

Furthermore, the second phase method could be refined. So far, we used GBMs on top of our feature engineering-based approach. A promising alternative could be a neural network (NN) approach that not only performs the scoring of selected items, but also incorporates the first phase item selection task in one single method. Extensive feature engineering will mostly likely still be essential for such a method to be effective. While NN approaches nowadays perform well also for certain types of tabular data, they often seem to be not at the same performance level yet when it comes to feature learning as they are for example on image recognition or natural language processing. Likewise, the principle of incorporating several other session-based or session-aware recommendation methods will probably still be an important factor, as different approaches learn different patterns in the data and thus provide a synergy.

Author Contributions JB and DJ wrote the main manuscript text. All authors reviewed the manuscript.

Funding Open access funding provided by University of Klagenfurt. No funds, grants, or other support was received. The authors have no relevant financial or non-financial interests to disclose.

Declarations

Conflict of interest The authors declare no competing interests.

Appendix A: List of engineered features

In the following, in Table 14 we provide a list of the features which we engineered, organized by the different groups with their motivations outlined in Sect. 4.3. Note that there can be multiple features by row, as evident in their description.

[9] We empirically confirmed the large potential by evaluating an additional idealized scenario in which all true positives are supplied in the first phase, which provides an upper bound for the potential of better first phase recommenders.

Table 14 List of the generated features mentioned in Sect. 4.3 on feature engineering

User–item–session specific features
The predicted ranks by both first phase recommenders and their combined ranking
The number of views, add-to-carts and purchases by the user for every relevant item in the current session up to the given time stamp (each as a separate feature)
The prior observation time of the item by the user in the current session
The category of the item the user last interacted with in a session (as a categorical feature)
The category of the item for which a prediction is made (as a categorical feature)
A binary indicator whether or not the category of the last interacted item is the same as the category of the item for which a prediction is made
An indicator whether the current item is identical to the item being predicted (excluded in the experiments if *true*)
Whether the item to be predicted is available at the respective time, i.e., in stock for fast delivery
The price of the item the user last interacted with
The price of the target item
The price difference between the target item and the last item the user interacted with
The relative price difference between the target item and the last item
User–item specific features
The time since a user first had an interaction with the future target item to be predicted (if there was any), and distinguished by interaction type
The time since a user last interacted with/viewed/added/bought a given item
The running count of interactions, the number of view, add-to-cart and purchase events of a user for every relevant item up to the current time for which a prediction is made
How many sessions ago the user last interacted with an item
The number of sessions since the user interacted with an item for the first time
The prior observation time of the item by the user in total
The price difference of the item to be predicted between the current session and the last session
User–category specific features
The time since the first interaction of a user with the category of an item to be predicted
The time since the last interaction with this category
The times since the first and the last view/add-to-cart and purchase events for any item of the target category (yielding 6 different features)
The cumulative number of interactions a user had in the past with items from the target category
The number of sessions since a user last interacted with items from the target category
The mean price of items the user viewed/bought from the category

Table 14 continued

User–category specific features
The price sensitivity of a user for buying in the given category (defined by difference in means, e.g., a higher positive value resembles that a user prefers cheaper products in a category)
The price sensitivity of a user for browsing in the given category of the item to be predicted
Session specific features
The time since the start of the current session
The number of observed actions by the user in the current session
Whether the last observed event was a view, add-to-cart or purchase event
The cumulative count of view, add-to-cart and purchase events of the user over all items in the given session, for a varying number of last observations
The weekday of the current session (may change if the session lies between two days)
The smoothed current time of the day
User specific features
The number of sessions the given user already had
The time since the last interaction observed for the given user
The time since the users' last view, add-to-cart and purchase
The running count of views/add-to-carts/purchases by a user, for different window lengths (e.g., 10 and 100, yielding 6 features)
The price sensitivity of the user, defined as the difference between the mean price of items all users selected and the mean price of items the current user selected
Quantiles of the hours of day with user activity
Statistics on the hours of day when the user made purchases (the median and quartiles)
The cumulative count of both view and either add-to-cart or purchase events for a user by every weekday, yielding 14 features used to construct two features that measure these numbers for the weekday of the current session
The cumulative number of unique items and unique categories the user interacted with
The running number of observations for the user in the last three months
Product category specific features (of the target item)
The cumulative number of unique users that interacted with the category of the candidate item
The mean price of items in the respective category
The mean price of items in the category which users selected
The running number of observations in the given category (all interactions in the past three months with respect to the current time of prediction)

Table 14 continued

Item specific features (of the target item)
The time since the first appearance of an item (i.e., any interaction type by any user)
The time since the last appearance of an item
The time an item was first viewed/added to a basket or bought by any user
How long ago such view, add-to-cart and purchase events happened last
The cumulative number of appearances of an item in the clickstream log data
The count of views, add-to-cart events and purchases of the given item in the past
Moving time window statistics for interactions with the given item in all sessions in recent times, distinguished between view, add-to-cart and purchase events. Their respective numbers are calculated within the last hour, day, week and last four weeks with respect to the current time stamp (i.e., 12 features in total). The specific window sizes can be adapted
The number of all observations made for an item in the last three months
The number of unique users that have interacted with the item
The price difference between the item price and the mean price of items in the respective category
The price difference between the item price and the mean price of items that were bought in the respective category
Sparse categorical features
Sparse categorical features of users, items and categories by their IDs as well as the most recent event type observed in the session

References

Adomavicius, G., Kwon, Y.O.: Improving aggregate recommendation diversity using ranking-based techniques. IEEE Trans. Knowl. Data Eng. **24**(5), 896–911 (2011)

Afchar, D., Melchiorre, A.B., Schedl, M., Hennequin, R., Epure, E.V., Moussallam, M.D.: Explainability in music recommender systems. (2022) arXiv preprint arXiv:2201.10528

Ben-Shimon, D., Tsikinovsky, A., Friedmann, M., Shapira, B., Rokach, L., Hoerle, J.: Recsys challenge 2015 and the YOOCHOOSE dataset. In Proceedings of the 9th ACM Conference on Recommender Systems, RecSys 2015, pp. 357–358 (2015)

Burke, R.: Hybrid recommender systems: survey and experiments. User Model. User-Adap. Inter. **12**(4), 331–370 (2002)

Chen, H., Lundberg, S.M., Lee, S.-I.: Explaining a series of models by propagating shapley values. Nature Commun. **13**(1), 4512 (2022)

Chen, S., Moore, J.L., Turnbull, D., Joachims, T.: Playlist prediction via metric embedding. In Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '12, pp. 714–722 (2012)

Gabriel De Souza, P.M., Jannach, D., Da-Cunha, A.M.: Contextual hybrid session-based news recommendation with recurrent neural networks. IEEE Access **7**, 169185–169203 (2019)

Garcin, F., Dimitrakakis, C., Faltings, B.: Personalized news recommendation with context trees. In Proceedings of the ACM Confererence on Recommender Systems, RecSys '13, pp. 105–112 (2013)

Geng, C., Wu, H., Fang, H.: Causality and correlation graph modeling for effective and explainable session-based recommendation. (2022) arXiv preprint arXiv:2201.10782

Hariri, N., Mobasher, B., Burke, R.: Context-aware music recommendation based on latent topic sequential patterns. In Proceedings of the Sixth ACM Conference on Recommender Systems, RecSys '12, pp. 131–131 (2012)

Hastie, T., Tibshirani, R., Friedman, J.: The Elements of Statistical Learning: Data Mining, Inference, and Prediction. Springer Science & Business Media, Berlin (2009)
Hidasi, B., Karatzoglou, A., Baltrunas, L., Tikk, D.: Session-based recommendations with recurrent neural networks. In Proceedings International Conference on Learning Representatinos, ICLR '16, (2016)
Hidasi, B., Quadrana, M., Karatzoglou, A., Tikk, D.: Parallel recurrent neural network architectures for feature-rich session-based recommendations. In Proceedings of the 10th ACM conference on recommender systems, pp. 241–248 (2016)
Liang, H., Chen, Q., Zhao, H., Jian, S., Cao, L., Cao, J.: Neural cross-session filtering: next-item prediction under intra- and inter-session context. IEEE Intell. Syst. **33**(6), 57–67 (2018)
Jannach, D., Jugovac, M.: Measuring the business value of recommender systems. ACM Trans. Manage. Inf. Syst. **10**(4), 1–23 (2019)
Jannach, D., Lerche, L., Jugovac, M.: Adaptation and evaluation of recommendations for short-term shopping goals. In Proceedings of the ACM Conference on Recommender Systems, RecSys '15, pp. 211–218 (2015)
Jannach, D., Ludewig, M., Lerche, L.: Session-based item recommendation in e-commerce: on short-term intents, reminders, trends and discounts. User Model. User-Adap. Inter. **27**(3), 351–392 (2017)
Jannach, D., Quadrana, M., Cremonesi, P.: Session-based recommendation. In: Ricci, F., Shapira, B., Rokach, L. (eds.) Recommender Systems Handbook. Springer, US (2021)
Kang, W.-C., McAuley, J.: Self-attentive sequential recommendation. In 2018 IEEE International Conference on Data Mining (ICDM), pp. 197–206 (2018)
Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.-Y.: Lightgbm: a highly efficient gradient boosting decision tree. Adv. Neural. Inf. Process. Syst. **30**, 3146–3154 (2017)
Kouki, P., Fountalis, I., Vasiloglou, N., Cui, X., Liberty, E., Al Jadda, K.: From the lab to production: a case study of session-based recommendations in the home-improvement domain. In Fourteenth ACM Conference on Recommender Systems, RecSys '20, pp. 140–149, (2020)
Latifi, S., Mauro, N., Jannach, D.: Session-aware recommendation: a surprising quest for the state-of-the-art. Inf. Sci. **573**, 291–315 (2021)
Li, J., Ren, P., Chen, Z., Ren, Z., Lian, T., Ma, J.: Neural attentive session-based recommendation. In Proceedings of the 2017 ACM on Conference on Information and Knowledge Management, CIKM '17, pp. 1419–1428 (2017)
Liu, Q., Zeng, Y., Mokhosi, R., Zhang, H.: STAMP: short-term attention/memory priority model for session-based recommendation. In Proceedings ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '18, pp. 1831–1839 (2018)
Ludewig, M., Jannach, D.: Evaluation of session-based recommendation algorithms. User Model. User-Adap. Inter. **28**(4–5), 331–390 (2018)
Ludewig, M., Mauro, N., Latifi, S., Jannach, D.: Empirical analysis of session-based recommendation algorithms. User Model. User-Adap. Inter. **31**(1), 149–181 (2021)
Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. In Advances in neural information processing systems, pp. 4765–4774 (2017)
Lundberg, S.M., et al.: From local explanations to global understanding with explainable ai for trees. Nature Mach. Intell. **2**(1), 56–67 (2020)
Misztal-Radecka, J., Indurkhya, B.: Bias-aware hierarchical clustering for detecting the discriminated groups of users in recommendation systems. Inf. Process. Manage. **58**(3), 102519 (2021)
Mobasher, B., Dai, H., Luo, T., Nakagawa, M.: Using sequential and non-sequential patterns in predictive web usage mining tasks. In Proceedings of IEEE International Conference on Data Mining, ICDM '02, pp. 669–672 (2002)
Phuong, T.M., Thanh, T.C., Bach, N.X.: Combining user-based and session-based recommendations with recurrent neural networks. In International Conference on Neural Information Processing, pp. 487–498. Springer, Berlin (2018)
Phuong, T.M., Thanh, T.C., Bach, N.X.: Neural session-aware recommendation. IEEE Access **7**, 86884–86896 (2019)
Quadrana, M., Cremonesi, P., Jannach, D.: Sequence-aware recommender systems. ACM Comput. Surv. **51**, 1–36 (2018)
Quadrana, M., Karatzoglou, A., Hidasi, B., Cremonesi, P.: Personalizing session-based recommendations with hierarchical recurrent neural networks. In proceedings of the Eleventh ACM Conference on Recommender Systems, pp. 130–137 (2017)

Ragno, R., Burges, C.J.C., Herley, C.: Inferring similarity between music objects with application to playlist generation. In Proceedings of the 7th ACM SIGMM International Workshop on Multimedia Information Retrieval, MIR '05, pp. 73–80 (2005)

Ren, P., Chen, Z., Li, J., Ren, Z., Ma, J., de Rijke, M.: Repeatnet: A repeat aware neural recommendation machine for session-based recommendation. In Proceedings of the AAAI Conference on Artificial Intelligence, pp. 4806–4813 (2019)

Roberts, C.V., Elahi, E., Chandrashekar, A.: On the bias-variance characteristics of lime and shap in high sparsity movie recommendation explanation tasks. (2022) arXiv preprint arXiv:2206.04784

Sánchez Rodríguez, J.A., Wu, J.-C., Khandwawala, M.: Two-stage session-based recommendations with candidate rank embeddings. In Fashion Recommender Systems, pp. 49–66. Springer, Berlin (2020)

Ruocco, M., Lillestøl Skrede, O.S., Langseth, H.: Inter-session modeling for session-based recommendation. In Proceedings of the 2nd Workshop on Deep Learning for Recommender Systems, DLRS 2017, pp. 24–31, (2017)

Sachdeva, N., Manco, G., Ritacco, E., Pudi, V.: Sequential variational autoencoders for collaborative filtering. In Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining, WSDM '19, pp. 600–608 (2019)

Shani, G., Heckerman, D., Brafman, R.I.: An MDP-based recommender system. J. Mach. Learn. Res. **6**, 1265–1295 (2005)

Song, B., Cao, Y., Zhang, W., Xu, C.: Session-based recommendation with hierarchical memory networks. In Proceedings of the 28th ACM International Conference on Information and Knowledge Management, pp. 2181–2184 (2019)

Song, Y., Lee, J.-G.: Augmenting recurrent neural networks with high-order user-contextual preference for session-based recommendation. (2018) arXiv preprint arXiv:1805.02983

Sun, F., Liu, J., Wu, J., Pei, C., Lin, X., Ou, W., Jiang, P.: BERT4Rec: sequential recommendation with bidirectional encoder representations from transformer. In Proceedings of the 28th ACM International Conference on Information and Knowledge Management, pp. 1441–1450 (2019)

Tavakol, M., Brefeld, U.: Factored MDPs for detecting topics of user sessions. In Proceedings of the 8th ACM Conference on Recommender Systems, RecSys '14, pp. 33–40 (2014)

Tavakol, M., Brefeld, U.: Factored mdps for detecting topics of user sessions. In Proceedings of the 8th ACM Conference on Recommender Systems, pp. 33–40 (2014)

Ukil, A., Marin, L., Jara, A.J.: When less is more powerful: shapley value attributed ablation with augmented learning for practical time series sensor data classification. Plos one **17**(11), e0277975 (2022)

Wang, M., Ren, P., Mei, L., Chen, Z., Ma, J., de Rijke, M.: A collaborative session-based recommendation approach with parallel memory modules. In Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '19, pp. 345–354 (2019)

Wang, S., Cao, L., Wang, Y., Sheng, Q.Z., Orgun, M.A., Lian, D.: A survey on session-based recommender systems. ACM Comput. Surv. **54**(7), 1–38 (2021)

Wu, S., Tang, Y., Zhu, Y., Wang, L., Xie, X., Tan, T.: Session-based recommendation with graph neural networks. In Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence, AAAI, pp. 346–353 (2019)

Ying, H., Zhuang, F., Zhang, F., Liu, Y., Xu, G., Xie, X., Xiong, H., Wu, J.: Sequential recommender system based on hierarchical attention network. In Proceedings of the 27th International Joint Conference on Artificial Intelligence, IJCAI'18, pp. 3926-3932. AAAI Press, (2018)

Yu, F., Zhu, Y., Liu, Q., Wu, S., Wang, L., Tan, T.: TAGNN: target attentive graph neural networks for session-based recommendation. In Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '20, (2020)

Yuan, F., Karatzoglou, A., Arapakis, I., Jose, J.M., He, X.: A simple convolutional generative network for next item recommendation. In Proceedings of the 12th ACM International Conference on Web Search and Data Mining, WSDM '19, pp. 582–590 (2019)

Zhu, N., Cao, J., Liu, Y., Yang, Y., Ying, H., Xiong, H.: Sequential modeling of hierarchical user intention and preference for next-item recommendation. In Proceedings of the 13th International Conference on Web Search and Data Mining, pp. 807–815 (2020)

Josef Bauer studied mathematics and is currently a PhD student at University of Klagenfurt, Austria. He worked on various machine learning problems, including recommender systems.

Dietmar Jannach is a Professor of Computer Science at University of Klagenfurt, Austria. He has worked on different areas of artificial intelligence, including recommender systems, model-based diagnosis, and knowledge-based systems. He is the leading author of a textbook on recommender systems and has authored more than hundred research papers, focusing on the application of artificial intelligence technology to practical problems.